

When a Git repository is created, the root directory of the repository contains a special directory named `.git`, which contains all the data required for Git's functioning. All the binary objects (commits, blobs and trees) are stored in the `.git/objects` sub-directory. When you add files into the repository, corresponding blobs are created. When you commit your change with a description, a commit object is created, and it references the current files in the working directory. When you modify a file, Git creates another copy of the file internally, and renames it as the SHA1 hash for that newly created file. Thus, Git handles content-addressable data. When some file is modified manually, Git will see the mismatch between the SHA1 of the file content and its name, and this makes it easier to preserve the integrity of the data.

The local repository

In Git, you have a full copy of the repository locally. Whatever changes you make are to the local repository. You can work offline without any connectivity to a master server and make commits to the local repository. To sync changes to the master server, you can add the remote repository and push changes into it, or sync your local repository with new changes in the remote repository.

The working directory and staging area

The working directory is the repository directory where files are checked out. Whenever you want to do a commit after making some modifications, unlike other version control systems, Git will not track the modifications made to all the files in the working directory. When you do a *Git commit*, it will look at the files added to the staging area. Only files in the staging area are considered for a commit. Basically, the staging area is a method by which you can selectively add files in the current working directory for a specific commit. Let's suppose there are 10 files in the directory. You made modifications to four files, but changes to two files belong to a particular feature addition, and other modifications belong to some other feature. You don't want to commit changes related to two features in a single commit, but changes related to one feature in one commit, and other changes in another commit. The staging area helps you to do that, as follows:

```
git add file1 file2 file3
```

...can be used to add files to the staging area, and then run *git commit* to add those changes in the staging area to a new commit. After the first commit, add the other files, and do the next commit. The *git add* command basically moves the added files to the staging area; only added files are considered for a commit.

Every object in Git can be referenced by the SHA1 hash (41 characters). Usually, you do not need to type 41 characters—for any reference, only the first five characters of the hash are required.

Branches and tags

By default, Git uses the branch named *master*, which is just like *trunk* in Subversion—but the concept of branches is different in Git. A branch can be a short-lived branch or long-term, according to Git concepts. Usually we use short-lived branches (called topic-branches) to develop new features; once the feature development is complete, you can merge it back to the main branch (*master*) and delete the topic-branch. Creating and deleting branches is never expensive in Git. It is part of the Git workflow to create and delete branches.

Every branch is referenced by a branch head (HEAD)—a pointer to the latest commit in that branch. When you make commits to a branch, the head is updated with the latest commit. The heads of branches are listed in the directory `.git/refs/heads/`:

```
$ cat .git/refs/heads/master
46bf1f4592a7f121438a16a9d5e28e16db239614
```

The contents of the master branch head file refer to the commit ID of the latest commit in the master branch.

Tags are very similar to branches—but the difference is that tags are immutable. Once you create a tag for a particular commit, even when you create a new commit, it will not be updated. Suppose you want to tag some particular commit as a version release, you create a tag and keep it.

You can refer to the latest commit in the current working branch using the shorthand 'HEAD'. You can also reference parent commits from the latest current commits using *HEAD-N* to reference the *n*th older commit from a particular moment. You can run *git show HEAD* to display information about the latest commit.

Fast-forward and conflicting merges

Merging is a common task when working with version-control systems. Many people may work on the same source code, each developing different portions of it. At some point, you will need to merge all the code into the main branch. Git has its own interesting ways of merging code. There are two terms associated with merging code: *fast-forward merge* and *conflicting merge*. Let us represent our Git repository as a series of commits, each having a parent commit, except for the first commit:

A-B-C-D

Here, A is the first commit and B, C and D are next commits. D is the HEAD commit (the latest). One of the developers (Dev-1) clones the repo to his local machine, and adds a few more commits. Let us represent his local repo as follows:

A-B-C-D-E-F

Dev-1 now needs to merge his changes back to the